Light Instruments User Guide

Thomas Röggla - CWI

August 5, 2026

Contents

1 Introduction **2**
1.1 Overview . . . 2
1.2 The Instruments . . . 2

2 Getting Started **3**
2.1 Connecting the Instruments . . . 3
2.2 Connecting the LED Controllers . . . 3
2.3 Setting up the Node Editor . . . 4

3 Node Setups **4**
3.1 Concepts . . . 4
3.1.1 Interacting with the Editor . . . 5
3.2 Setups for Debugging & Exploration . . . 6
3.3 Device Filtering . . . 6
3.4 Sending commands . . . 6

4 Node Documentation **8**
4.1 Action . . . 8
4.1.1 Command . . . 8
4.1.2 Script . . . 8
4.2 Display . . . 8
4.2.1 Graph . . . 8
4.2.2 Log . . . 9
4.2.3 Statistics . . . 9
4.3 Input & IO . . . 9
4.3.1 Button . . . 9
4.3.2 CSV Writer . . . 9
4.3.3 Function Generator . . . 9
4.3.4 Serial Input . . . 9
4.3.5 Serial Output . . . 9
4.3.6 Simulate . . . 9
4.3.7 Value . . . 9
4.4 Layout . . . 9
4.4.1 Frame . . . 9
4.5 Math & Logic . . . 10
4.5.1 Boolean . . . 10
4.5.2 Compare . . . 10
4.5.3 Counter . . . 10
4.5.4 Cumulative Sum . . . 10
4.5.5 Delay . . . 10
4.5.6 Edge Trigger . . . 10

4.5.7 Gate . . . 10
4.5.8 Hysteresis . . . 10
4.5.9 Math . . . 11
4.5.10 Peak Detection . . . 11
4.5.11 Timer . . . 11
4.5.12 Toggle . . . 11
4.6 Processing . . . 11
4.6.1 Clamp . . . 11
4.6.2 Combine RGB . . . 11
4.6.3 Deadband . . . 11
4.6.4 Derivative . . . 11
4.6.5 Device Filter . . . 12
4.6.6 Envelope Follower . . . 12
4.6.7 Map Range . . . 12
4.6.8 Median Filter . . . 12
4.6.9 Moving Average . . . 12
4.6.10 Quantize . . . 12
4.6.11 Rate . . . 12
4.6.12 Smooth . . . 12

5 Light Instrument Controller Commands 12
5.1 General Commands . . . 13
5.1.1 `set` . . . 13
5.1.2 `setColor` . . . 13
5.1.3 `setBrightness` . . . 13
5.1.4 `setLED` . . . 13
5.1.5 `stop` . . . 13
5.2 Animation Commands . . . 13
5.2.1 `rainbow` . . . 13
5.2.2 `glitter` . . . 13
5.2.3 `pulse` . . . 14
5.2.4 `comet` . . . 14
5.2.5 `breathe` . . . 14
5.2.6 `fire` . . . 14

1 Introduction

Welcome to the Light Instruments project. This guide will help you understand how to use and configure the various instruments and how to use the node editor to design lighting setups and have them light up the LED strips.

1.1 Overview

The project consists of several ESP32-S3 based instruments that communicate wirelessly via ESP-NOW, LED controllers that also communicate wirelessly, a receiver that plugs into your computer and a node-based editor to process the data coming from the instruments and transform it into commands for the LED controllers.

1.2 The Instruments

At this point in time, there exist the following instruments:

- *Key Instrument*: An instrument containing several buttons, which when pressed or released send a single signal about the button status to the receiver.
- *Maracas*: Sends a single signal to the receiver when movement around a specific axis is detected.

- *IMU Maracas*: Comes in the same form factor as the Maracas and works in the same fashion, but uses an IMU (inertial motion unit) to detect activity. This offers activity detection around any axis and better noise resistance.
- *Rainstick*: Contains an internal infrared barrier, which detects the passing of the percussive media inside the instrument through it and sends a corresponding signal to the receiver.
- *Touch Instrument*: Contains three touch surfaces, which when approached or touched by a hand sends a continuous signal identifying the amount of touch detected. Depending on initial calibration, the surfaces may not only react to touch, but also proximity of a hand.
- *Percussion Instrument*: The percussion instrument detects vibration using a piezoelectric element. When the surface of the instrument is struck using a mallet or hand and the detector detects the resulting vibration, a continuous signal is sent to the receiver. The sensitivity can be tuned by adjusting the 100 kOhm potentiometer on the circuit board.

2 Getting Started

First, plug the receiver into your computer using a USB cable. Then, you can start setting up the instruments and the LED controllers.

2.1 Connecting the Instruments

The instruments are powered by 1000 mAh Lithium cells. Open the battery compartments, locate the battery and battery connector. Plug the battery into the connector to activate it. The *Maracas* and *IMU Maracas* have a blue power switch located on the circuit port. In this case, the battery can remain plugged in at all times and the instrument can be turned on or off using the switch.

Now, observe the circuit board. An orange LED should blink twice immediately after gaining power. This signifies the start of the internal setup process. After at most 10 seconds, the same orange LED should turn on permanently, indicating that the instrument successfully discovered the receiver and is ready to transmit data. If the LED does not turn on, verify that the receiver is plugged into a USB port and the two are within wireless communication range. The communication range should be at about 200m with clear single of sight, but might drop to 20-40m with obstacles an in indoor environments. Note that communication efficiency is also affected in areas with increased wireless (namely WiFi) activity.

Note about the touch instrument: During boot, the touch instrument goes through a calibration procedure, adjusting the sensitivity of the touch surfaces based on environmental factors. Thus, when plugging in the instrument, try and avoid getting close to the touch surfaces with your hands or any other parts as this will influence the calibration procedure and lead to erroneous baseline calibration, affecting accurate touch detection during operation.

2.2 Connecting the LED Controllers

The setup procedure for the LED controllers is largely the same as for the instruments. The controllers can be either powered through the USB-C port or the 5V screw terminals on the back of the circuit board. When using the screw terminals, verify correct polarity and voltage before connecting to power.

Before connecting to power, also make sure the desired number of LED strips are connected to the four LED strip ports on the circuit board.

Analogous to the instruments, the onboard orange status LED will blink twice to indicate the start of the boot process. Then, after at most ten seconds, the LED will turn on permanently to indicate successful connection to the receiver. If this is not the case, make sure the receiver is plugged in and within wireless communication range.

2.3 Setting up the Node Editor

In order to read data from the instruments and design your own light setups, to be displayed using the LED controllers, after plugging the receiver into your computer, start the *AMPLIFY Light Instrument Editor*. In the default setup, you will see two nodes on the main canvas: *Serial Input* and *Serial Output*.

To get started, in the *Serial Input* node, select the name of the receiver from the dropdown and press *Connect*. On macOS this will be something like `/dev/cu.usbmodem101`, while on Windows it will be something like `COM1`. Upon successful connection, the *Serial Output* node should display the string `Connected`.

For a quick initial test of the LED controllers, find the command bar at the bottom of the window. From the command dropdown select the command `rainbow` and hit the *Send* button. Now all LED strips should display an animated rainbow pattern. Select the command `stop` and hit *Send* to stop the animation.

You can also verify presence and active communication with the controllers and instruments by locating the activity panel in the top left. This will display the names of all devices which the receiver was able to communicate, separated between instruments and LED controllers. Note that it might take a few seconds for this panel to be displayed. Names of devices from which no data was received recently will slowly fade out.

3 Node Setups

3.1 Concepts

The Light Instrument Editor uses a node-based interface to design lighting behaviors. Instead of writing code, you create a "node setup" (also known as a graph or patch) by placing functional blocks on a canvas and connecting them with lines.

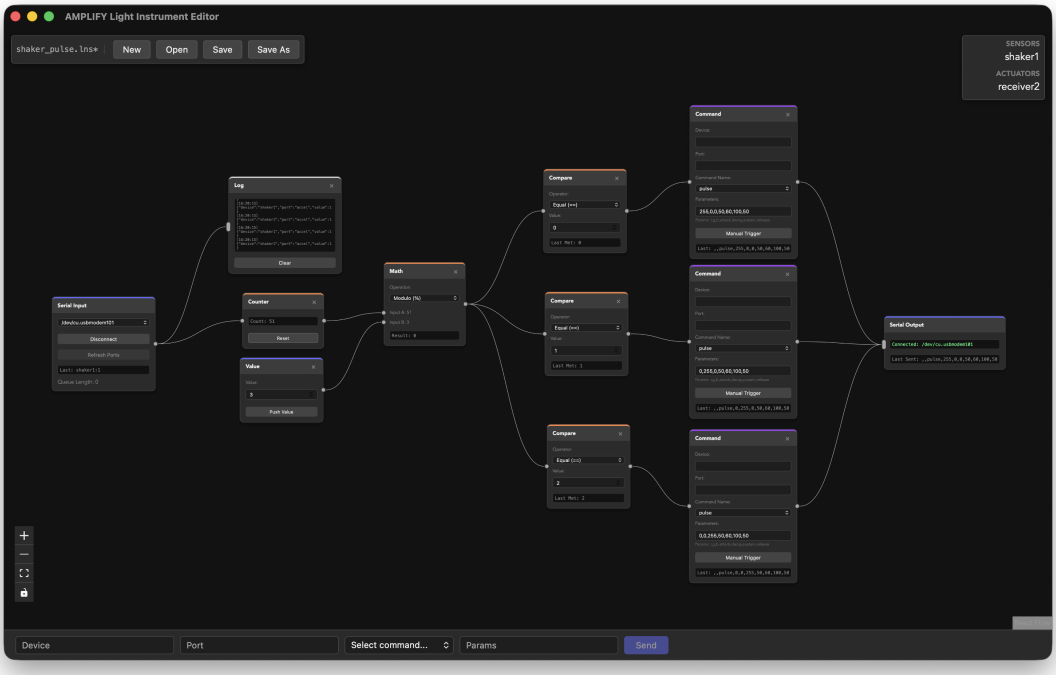

Figure 1: A basic node setup showing data flow from a Serial Input node to a Serial Output node via several Processing nodes.

Data flows from left to right: it starts at an **Input** node (like the physical serial port or a file simulation), passes through various **Processing** or **Math & Logic** nodes that transform the signals, and finally reaches an

Output node which sends commands to your LED controllers or logs data to a file. This visual approach allows you to quickly experiment with different sensor-to-light mappings without needing to recompile firmware.

3.1.1 Interacting with the Editor

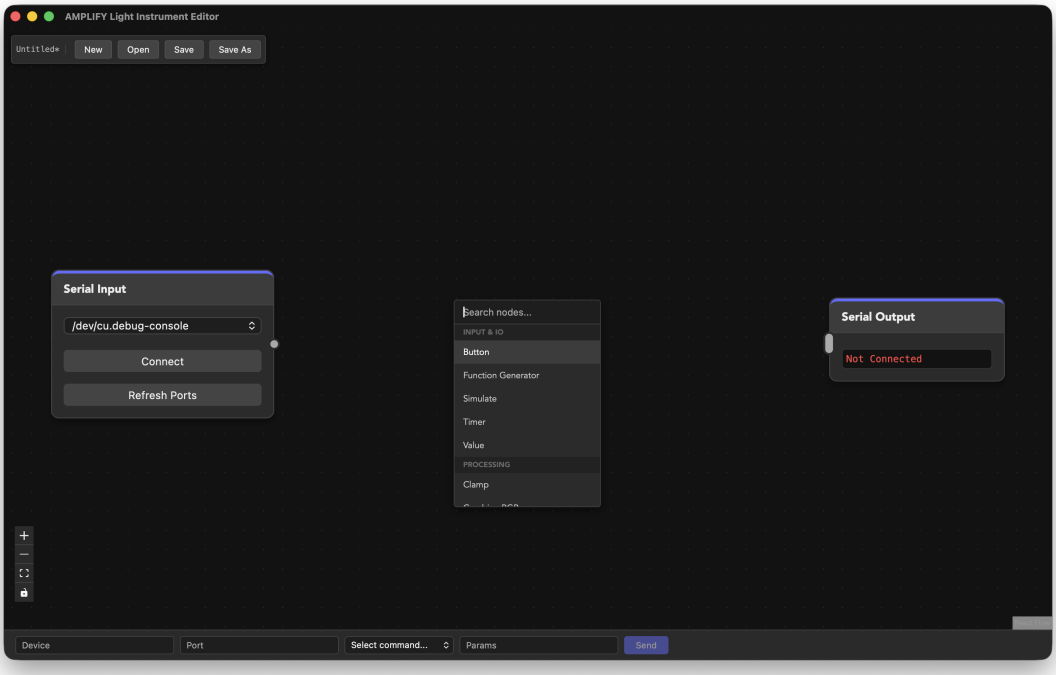

Figure 2: The context menu for adding nodes is activated by pressing Shift+A

To build your setup, you can interact with the editor using the following commands:

- **Adding Nodes**: Press `Shift+A` to open the search menu. Type the name of the node you want to add and press `Enter`, or click on the desired node in the list.
- **Deleting Nodes**: Click the "×" button in the top-right corner of the node's header.
- **Creating Connections**: Click and drag from an output handle (the circular socket on the right side of a node) to an input handle (on the left side of another node).
- **Deleting Connections**: Click on the connection line to select it, then press `Backspace` or `Delete` on your keyboard.

You can route the output of any node to the input of as many nodes as required. Most nodes only accept a single incoming connection. Some nodes, however, accept multiple inputs, e.g. the *Log* node or the *Serial Output* node. This is indicated by an elongated input socket.

You can open existing setups, save the current one or start with a fresh canvas using the buttons in the menu panel located in the top left of the canvas or the application menu.

Move the canvas by clicking it with your left mouse button and dragging it around. In the same fashion, you can move nodes around by clicking and dragging on their title bar. Zooming can be achieved by scrolling your mouse or using the pinch to zoom gesture on a trackpad. Alternatively, you can zoom in and out by using the `+` and `-` buttons in the toolbar located in the bottom left of the canvas. This toolbar also allows you to zoom the current setup to fit the screen or lock the entire canvas to prevent modification.

Check the `samples/` directory for a few example setups which show off the basic features of the node editor.

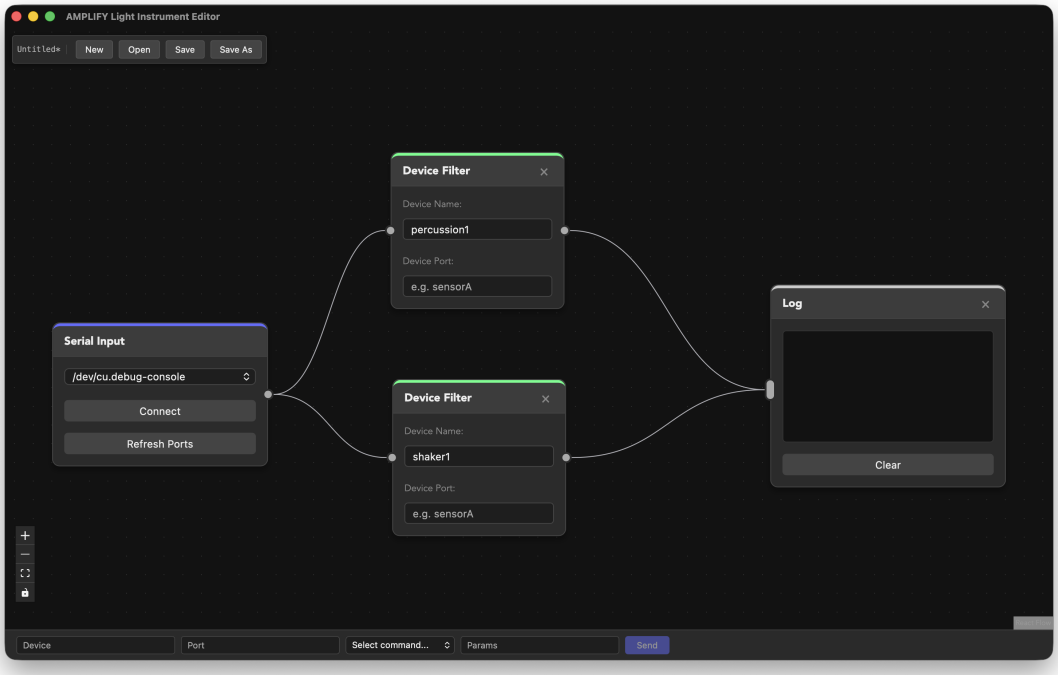

Figure 3: The Log node accepts multiple inputs, indicated by an elongated input socket

3.2 Setups for Debugging & Exploration

When building a new setup or troubleshooting a sensor, it can be useful to visualize the raw data arriving from the instruments. The editor provides two primary nodes for this purpose:

- **Log Node**: Displays a scrollable history of every data packet received on its input, complete with device names, ports, and timestamps. It is useful for verifying that a device is sending data at the expected rate.
- **Graph Node**: Renders a real-time line chart of incoming numeric values. This is invaluable for visualizing signal noise, verifying calibration and tuning thresholds for logic nodes like *Peak Detection* or *Compare*.

A common debugging pattern is to branch your signal and connect it to a *Graph* or *Log* node in parallel with your main processing logic.

3.3 Device Filtering

In a large setup with multiple instruments, the receiver's serial stream will contain data from many different sources. To prevent signals from interfering with each other, you can use the **Device Filter** node to direct data streams.

By configuring the *Device* and *Port* fields on this node, you can create a dedicated stream for a specific sensor. For example, if you only want to react to the "percussion1" instrument, you would set the device filter to "percussion1". Only packets matching your filter will be passed through to the output handle, allowing you to build modular logic for each individual instrument in your setup.

3.4 Sending commands

To bridge the gap between sensor data and lighting effects, you use the **Command** node. This node acts as a translator: when it receives a trigger signal on its input, it emits a formatted command packet that the

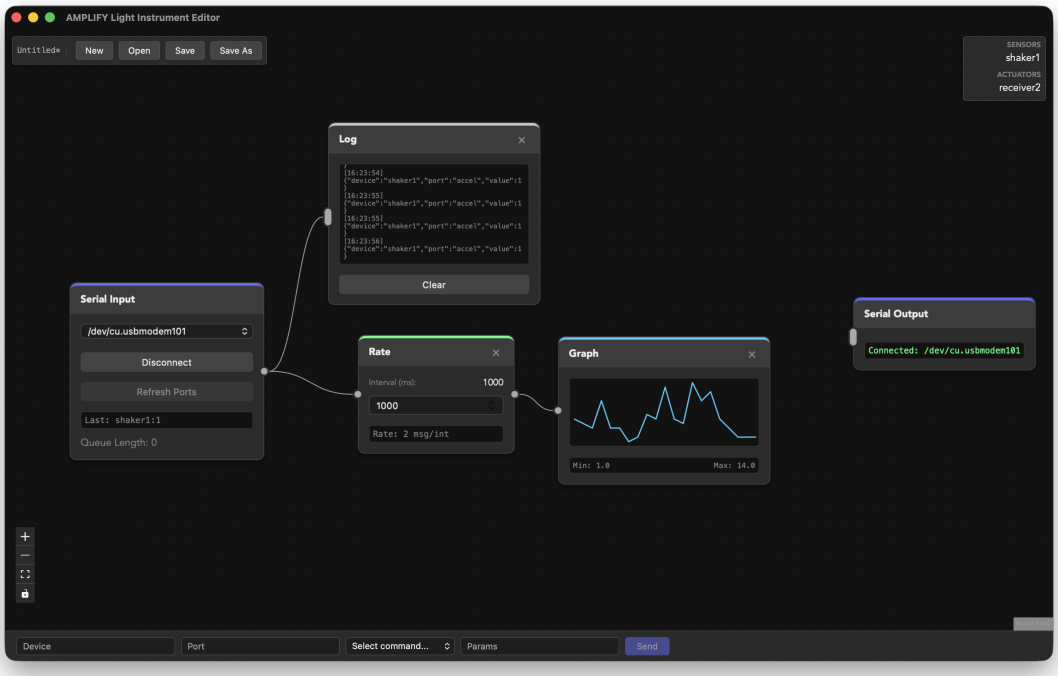

Figure 4: Debugging setup with all incoming messages being displayed in a Log node and a Graph node displaying the current rate of incoming messages per second through the use of the Rate node

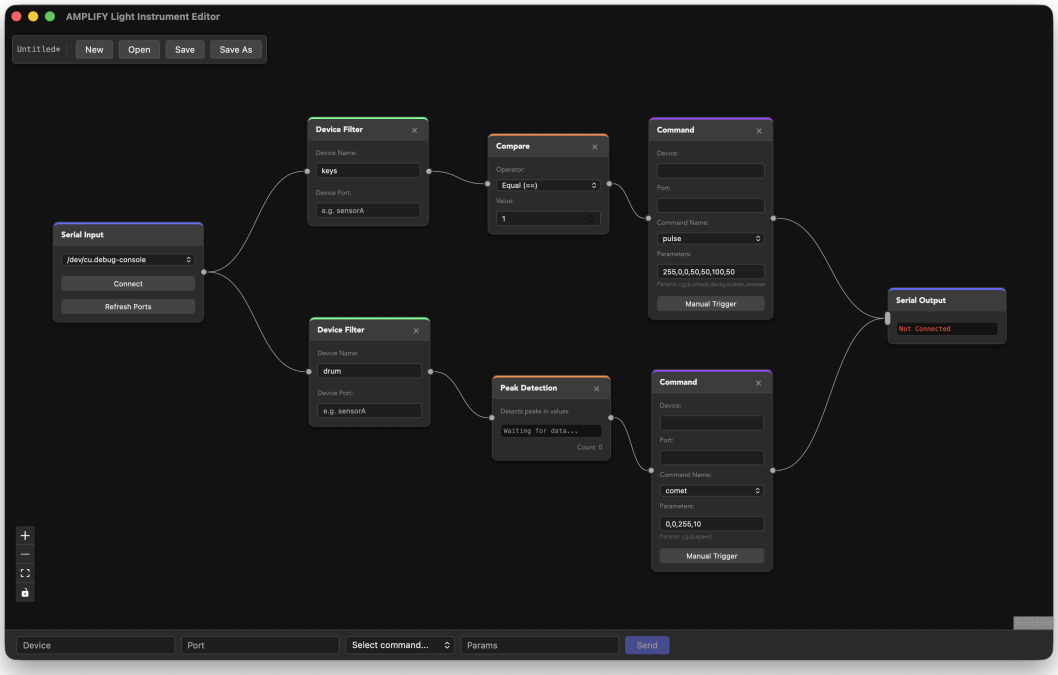

Figure 5: Two device filter nodes redirect the data streams in separate paths for the key instrument and the drum instrument, triggering different commands

LED controllers understand. Note that the *Serial Output* node only accepts connections from a *Command* or *Script* node.

In the *Command* node, you can select from a range of animations and static effects (like `set`, `pulse`, or `rainbow`) defined in the controller firmware. You can specify a target device and port, or leave them blank to broadcast the command to all controllers.

To assist with configuration, a **parameter hint** appears below the input field whenever a command is selected, showing the expected format and required values (e.g., `r,g,b,speed`). You can enter static values directly into the parameter field. For dynamic effects, you can use the `#` character as a placeholder; this character will be automatically replaced by the numeric value of the incoming signal, allowing sensor intensity to directly control parameters like brightness, speed, or color.

For testing, commands can be triggered by pressing the *Manual Trigger* button on the corresponding *Command* node.

4 Node Documentation

This section provides a description of all available nodes in the node-based editor, including their functions, inputs, and outputs.

4.1 Action

4.1.1 Command

Converts a trigger signal into a structured command packet for the serial output.

- **Input**: Any signal
- **Output**: Command packet `{device, port, command, value}`
- **Note**: Use "#" in parameters to inject incoming value.

4.1.2 Script

Executes a sequence of commands and delays.

- **Input**: Any signal
- **Output**: Command packets `{device, port, command, value}`
- **Format:** `command 'device' 'port' 'params'`
- **Delay:** `delay [ms]`

Example:

```
breathe 'receiver1' 'LED1' '255,0,0,20'
delay 500
stop '' '' ''
```

This will send a `breathe` command to `receiver1` port `LED1`, which causes a breathe animation with color red and a speed of 20 BPM to start. Then, the script waits for 500ms and then issues a `stop` command (empty strings for device and port send the command to all devices/port).

4.2 Display

4.2.1 Graph

Visualizes incoming numeric data on a real-time line chart.

- **Input**: Numeric value

4.2.2 Log

Displays a scrollable history of incoming data packets with timestamps.

- **Input**: Any data

4.2.3 Statistics

Maintains a live count of received packets grouped by device name.

- **Input**: Structured packet

4.3 Input & IO

4.3.1 Button

Interactive button that emits signals on press and release (momentary) or alternates state (toggle).

- **Output**: Numeric signal (0 or 1)

4.3.2 CSV Writer

Saves incoming data to a CSV file with timestamps.

- **Input**: Any data packet

4.3.3 Function Generator

Generates periodic waveforms (Sine, Square, Triangle, Sawtooth) at a set frequency.

- **Output**: Periodic numeric signal (0 to 1)

4.3.4 Serial Input

Interfaces with a physical serial port to receive raw data packets.

- **Output**: Structured packet `{device, port, value}`

4.3.5 Serial Output

Sends formatted command packets to the connected serial port.

- **Input**: Command packet `{device, port, command, value}`

4.3.6 Simulate

Reads recorded sensor data from a file and streams it into the editor.

- **Output**: Structured packet `{device, port, value}`

4.3.7 Value

Provides a static numeric value that can be manually pushed or emitted on connection.

- **Output**: Numeric value

4.4 Layout

4.4.1 Frame

A visual grouping component used to organize and label collections of nodes.

- **Functional Details**: (No functional inputs or outputs)

4.5 Math & Logic

4.5.1 Boolean

Performs logical operations (AND, OR, XOR, NOT) on two boolean inputs (non-zero is true).

- **Inputs**: A, B
- **Output**: 1 or 0

4.5.2 Compare

Compares input data against a threshold using mathematical operators.

- **Input**: Numeric value
- **Output**: Filtered numeric value

4.5.3 Counter

Increments a internal counter for every message received and emits the total.

- **Input**: Any signal
- **Output**: Current count

4.5.4 Cumulative Sum

Sums incoming numeric values over an infinite or sliding window buffer.

- **Input**: Numeric value
- **Output**: Current sum

4.5.5 Delay

Emits received events after a specified delay.

- **Input**: Any signal
- **Output**: Delayed signal

4.5.6 Edge Trigger

Detects rising or falling transitions in a signal and emits a single impulse.

- **Input**: Numeric signal
- **Output**: Impulse (1)

4.5.7 Gate

Allows or blocks a data stream based on a separate control signal.

- **Inputs**: Signal, Control
- **Output**: Signal (if control is non-zero)

4.5.8 Hysteresis

Uses two thresholds to provide stable on/off switching and prevent jitter.

- **Input**: Numeric value
- **Output**: 1 or 0

4.5.9 Math

Performs arithmetic operations (+, -, *, /, %) on two numeric inputs.

- **Inputs**: A, B
- **Output**: Calculation result

4.5.10 Peak Detection

Identifies local maxima (peaks) in a numeric stream and emits a trigger signal.

- **Input**: Numeric value
- **Output**: Trigger impulse

4.5.11 Timer

Emits periodic pulses at a fixed interval.

- **Output**: Pulse signal

4.5.12 Toggle

Alternates between 1 and 0 every time it receives an input pulse (Flip-Flop).

- **Input**: Any signal
- **Output**: 1 or 0

4.6 Processing

4.6.1 Clamp

Restricts the incoming signal to be within a minimum and maximum range.

- **Input**: Numeric value
- **Output**: Clamped value

4.6.2 Combine RGB

Combines three numeric inputs into a string of format "r,g,b". This is useful for merging inputs from three different sources into a color for use in the *Command* node.

- **Inputs**: R, G, B
- **Output**: String "r,g,b"

4.6.3 Deadband

Ignores small fluctuations in the signal within a specified threshold of the last value.

- **Input**: Numeric value
- **Output**: Filtered value

4.6.4 Derivative

Calculates the rate of change (velocity) of the incoming signal.

- **Input**: Numeric value
- **Output**: Delta value

4.6.5 Device Filter

Only allows packets from a specific device and/or port to pass through.

- **Input**: Structured packet
- **Output**: Filtered packet

4.6.6 Envelope Follower

Tracks the peak level of a signal with configurable attack and release times.

- **Input**: Numeric value
- **Output**: Envelope value

4.6.7 Map Range

Linearly rescales values from one range to another (e.g. 0-1023 to 0-255).

- **Input**: Numeric value
- **Output**: Scaled value

4.6.8 Median Filter

Removes spike noise by outputting the median of a sliding window of values.

- **Input**: Numeric value
- **Output**: Filtered value

4.6.9 Moving Average

Smooths signal by averaging values over a sliding window.

- **Input**: Numeric value
- **Output**: Averaged value

4.6.10 Quantize

Snaps incoming values to the nearest multiple of a set step size.

- **Input**: Numeric value
- **Output**: Quantized value

4.6.11 Rate

Measures the frequency (messages per second) of incoming data packets.

- **Input**: Any data
- **Output**: Frequency (Hz)

4.6.12 Smooth

Applies exponential smoothing to the data stream to reduce jitter.

- **Input**: Numeric value
- **Output**: Smoothed value

5 Light Instrument Controller Commands

This chapter describes the available commands that can be sent to the Light Instrument controllers.

5.1 General Commands

5.1.1 `set`

Sets a static color and brightness for the targeted port.

- **Parameters**: `r,g,b,brightness`
- **Behavior**: Updates the color of a running animation if one is active. If no animation is running, it sets the strip to a solid color and updates the strip's base brightness. All values have to be between 0-255.

5.1.2 `setColor`

Sets a static color for the targeted port.

- **Parameters**: `r,g,b`
- **Behavior**: Updates the color of a running animation if one is active. If no animation is running, it fills the strip with the specified color. Values have to be between 0-255.

5.1.3 `setBrightness`

Updates the base brightness of the targeted port.

- **Parameters**: `brightness` (0-255)
- **Behavior**: Adjusts the intensity of the LEDs without interrupting any currently running animation.

5.1.4 `setLED`

Sets a specific range of LEDs to a certain color.

- **Parameters**: `r,g,b,offset,numleds`
- **Behavior**: Sets `numleds` starting from `offset` to the specified RGB color. This command **interrupts and stops** any currently running animation on the targeted port to prevent the animation from overwriting the manual changes. Values for `r,g,b` are 0-255. `offset` and `numleds` are clamped to the strip's length.

5.1.5 `stop`

Stops all activity on the targeted port.

- **Parameters**: None
- **Behavior**: Halts any active animation and turns off all LEDs for the specified port.

5.2 Animation Commands

All animations run independently per port and can be updated in real-time using `set` or `setColor`. For a detailed description of each animation and its parameters, see animations.md.

5.2.1 `rainbow`

Starts a cycling rainbow animation.

- **Parameters**: `deltaHue` (optional)
- **Default**: 5
- **Behavior**: Gradually cycles colors across the strip. A higher `deltaHue` creates a more "compressed" rainbow.

5.2.2 `glitter`

Starts a sparkling glitter animation over a background color.

- **Parameters**: `r,g,b,duration`

- **Optional**: `duration` can be omitted (defaults to 0).
- **Behavior**: Randomly flashes white pixels over the specified background color. If `duration` is `0`, the animation runs indefinitely. The duration is given in milliseconds.

5.2.3 `pulse`

Starts an ADSR (Attack, Decay, Sustain, Release) pulse animation.

- **Parameters**: `r,g,b,attack,decay,sustain,release`
- **Behavior**: Smoothly pulses the color through four stages of timing (in milliseconds).

5.2.4 `comet`

Starts a moving comet effect.

- **Parameters**: `r,g,b,speed`
- **Behavior**: Sends a "comet" of color down the strip. The `speed` parameter determines how quickly the tail fades (in milliseconds).

5.2.5 `breathe`

Starts a smooth breathing/pulsing animation.

- **Parameters**: `r,g,b,bpm`
- **Behavior**: Pulses the color intensity following a sine wave at the specified Beats Per Minute (BPM).

5.2.6 `fire`

Starts an organic fire flicker effect.

- **Parameters**: `r,g,b,intensity`
- **Behavior**: Simulates a flickering fire using a heat-map algorithm based on the specified color and spark intensity (0-255).